Name: _____

Reg. No: _____

Ghulam Ishaq Khan Institute of Engineering Sciences & Technology

Faculty of Computer Science and Engineering
AI407L Lab Final Exam - Spring 2026

Instructor: Mr. Muhammad Naeem, M. Asadullah Amin Total Marks = 100

Course Code: AI407L **Program: BSAI**

Course Title: **AI Capstone Project Lab**

Date: _____

Course Instructor Signature: _____

Vetted Faculty Signature: _____

HOD Signature: _____

Instructions:

1. The examination consists of two parts: Part A and Part B. Both parts are mandatory.

2. Your system must be executable and demonstrably functional. Hardcoded outputs, mocked responses, or simulated behavior without actual pipeline execution will not be accepted.

3. Use meaningful variable names, function names, and file names that reflect system purpose. Poor naming reduces clarity and may affect grading.

4. This is an individual assessment. Collaboration or code sharing is strictly prohibited.

5. A proper technical report must accompany your submission. Poor documentation, even with working code, will affect marks.

6. Late submissions will not be entertained.

7. Missing files or incomplete components will receive zero for that section.

8. A Viva will be conducted after submission. You must be able to explain every line of code you submit.

PART A

Drift Monitoring & Feedback Loops [40 Marks] [CLO-3/ GA-5/P5]

Objective

Over Labs 1–11, you built, secured, evaluated, and containerized your Agentic AI System. The final missing piece is Post-Deployment Monitoring. In this part, you will **Assemble** a simple feedback collection and analysis layer on top of your existing project..

Requirements

1. Feedback Collection: Add a feedback option after each response: (10 Marks)

- 👍 Good
- 👎 Bad

2. Logging: Store interactions in a simple file: (10 Marks)

Acceptable formats:

- JSON
- CSV
- SQLite

Minimum fields:

- user_input
- agent_response
- feedback

3. Basic Analysis: Create a script called analyze.py that does the following: (10 Marks)

- Count total responses
- Count negative feedback
- Print Top 3 failed queries

4. Improvement: Identify an issue, fix it, and show before vs after results: (10 Marks)

- Identify 1 common issue
- Fix it (prompt or retrieval)
- Show before vs after result

Deliverables (Part A)

1. feedback_log.json / .db
2. analyze.py
3. analysis_report.md
4. improvement_demo.md

Rubric (Part A)

Criteria	Marks	Full Marks Requirements
Feedback System	10	Feedback option (thumbs up/down) is added after each agent response and functions correctly.
Logging	10	Interactions are stored in JSON, CSV, or SQLite with all required fields: user_input, agent_response, feedback.
Analysis	10	analyze.py counts total responses, counts negative feedback, and prints the Top 3 failed queries.
Improvement	10	One common issue is identified, fixed (via prompt or retrieval), and a clear before vs after result is shown.

PART B

Self-RAG Agent — Independent Scenario [60 Marks] [CLO-2/ GA-4/P4]

Background: What is Self-RAG?

Standard RAG pipelines have two fundamental weaknesses. First, they always retrieve, even when the LLM already knows the answer or when retrieval adds noise. Second, they blindly trust whatever documents are retrieved, generating responses even from irrelevant context.

Self-RAG (Self-Reflective Retrieval-Augmented Generation) addresses these issues by introducing reflection checkpoints into the pipeline. Instead of following a fixed retrieve-then-generate path, a Self-RAG agent makes adaptive decisions at each stage:

1. Should I retrieve at all? Some queries (greetings, general knowledge) don't need document retrieval. Forcing retrieval in such cases introduces noise and slows down the system.

2. Is what I retrieved actually useful? After retrieval, the agent evaluates whether the documents are relevant. Irrelevant documents are discarded rather than being blindly fed to the generator.

3. Is my generated answer faithful to the evidence? After generating a response, the agent checks whether it actually stuck to the facts. If the response contains hallucinated information, the agent self-corrects.

Your task is to design and implement a complete Self-RAG pipeline using LangGraph.

The Scenario

You are **constructing** a University Course Advisory Agent for XYZ National University. Students interact with this agent to get answers about courses, prerequisites, credit hours, semester schedules, grading policies, fees, and faculty. You are provided with the university's official catalog documents (5 PDF files). The agent must use Self-RAG to ensure it retrieves only when needed, never trusts irrelevant documents, and never hallucinates critical academic information..

Provided Documents (in the data/ folder):

1. CS_Department_Catalog.pdf : 12 Computer Science courses (undergraduate + graduate).
2. EE_Department_Catalog.pdf: 8 Electrical Engineering courses.
3. BBA_Department_Catalog.pdf: 7 Business Administration courses.
4. University_Academic_Policies.pdf: Grading scales, GPA rules, attendance, fees, calendar, withdrawal policies, etc.
5. Faculty_Directory.pdf : Names, departments, specializations, emails, and offices of all faculty.

Do NOT create your own data: use the provided files as your knowledge base.

Requirements

Your Self-RAG agent must satisfy all of the following requirements.

1) Adaptive Retrieval: The agent decides whether a query needs retrieval from the knowledge base before searching. (10)

- If the query is conversational, a greeting, or can be answered from general knowledge (e.g., "Hi there", "What does GPA stand for?"), the agent must NOT search the vector database. It should answer directly.
- If the query is about specific university information (courses, prerequisites, policies, faculty, fees), the agent must retrieve from the knowledge base before answering.
- The retrieval decision and its reasoning must be visible in the execution trace.

2) Relevance Grading: After retrieval, the agent evaluates whether retrieved documents actually answer the user's question. (10)

- Each retrieved document must be individually assessed as relevant or irrelevant to the specific query.
- Irrelevant documents must be discarded and not used for generation.
- If all retrieved documents are irrelevant, the agent must not generate from them. Instead, it should fall back to web search to find the answer from an external source.

3) Web Search Fallback: When the knowledge base fails, the agent falls back to web search for an answer. (5)

- Use any search tool or API (Tavily, SerpAPI, DuckDuckGo, etc.).
- The web search results should replace the irrelevant documents as the generation context.

4) Hallucination Self-Check: After generating a response, the agent verifies the answer is grounded in the retrieved context. (10)

- The agent must check whether the generated response contains claims that are NOT supported by the source documents or web results.
- If a hallucination is detected, the agent must regenerate the response (retry), not simply pass it through.
- There must be a maximum retry limit (e.g., 2–3 attempts). If the agent still fails after the retry limit, it should respond with a clear disclaimer that it could not verify the information.

5) Knowledge Base Setup: Ingest the provided documents and build a searchable vector knowledge base. (10)

- Process all 5 provided PDF files.
- Apply a chunking strategy that respects the structure of the documents (e.g., keep course descriptions together, don't split a policy rule mid-sentence).
- Add meaningful metadata to each chunk (e.g., department, document type, course level).
- Index the chunks into a vector database (ChromaDB, FAISS, or Pinecone).

6) LangGraph Implementation: The entire pipeline must be implemented as a LangGraph StateGraph. (5)

- You must define appropriate state variables, nodes, and conditional edges.
- All tools must use the @tool decorator with Pydantic validation and descriptive docstrings.
- The graph must be compilable and runnable.

Testing Requirements (10)

You must demonstrate that your agent correctly handles all Self-RAG decision paths. Run at least 5 test cases that cover the following scenarios:

#	Scenario That Must Be Covered	Example Query (use your own)
1	A query where retrieval is NOT needed	e.g., a greeting, or a question the LLM can answer from general knowledge
2	A query where retrieval IS needed and the documents are relevant	e.g., a question about a specific course's prerequisites or credit hours
3	A query where retrieval IS needed but the documents are irrelevant (triggering web search fallback)	e.g., a question about something not covered in the catalog at all
4	A query where the hallucination check fails and the agent regenerates	e.g., engineer a scenario where the initial generation includes unsupported claims
5	Your own creative test case	Design any additional query that exercises your agent's logic

For each test case, document:

- The query you used.
- The expected behavior (which path should the agent take and why).
- The actual behavior (which path did the agent take — show the execution trace).
- The agent's final response.

Traces can be console logs, structured JSON logs, or LangSmith traces. Save all results in evaluation_results.md.

Deliverables (Part B)

1. self_rag_agent.py: Main entry point to run the agent interactively.
2. graph.py: Your LangGraph StateGraph implementation with all nodes, edges, and routing logic.
3. tools.py: Tool definitions with @tool decorators and Pydantic validation.
4. evaluation_results.md: All 5+ test cases with queries, expected behavior, actual traces, and final responses.

Assessment Rubric — Part B

Criteria	Marks	Full Marks Requirements
Knowledge Base	10	All 5 PDFs ingested; chunking respects document structure; metadata is meaningful; vector store works.
Adaptive Retrieval	10	Agent correctly skips retrieval for general queries and retrieves for domain-specific ones.
Relevance Grading	10	Documents are individually graded; irrelevant docs discarded; web fallback triggers when all docs fail.
Hallucination Check	10	Self-check detects unsupported claims; retry mechanism works; retry limit enforced.
LangGraph & Tools	5	Graph compiles and runs; tools have @tool decorators and Pydantic validation; code is clean.
Web Search Fallback	5	Web search triggers correctly and results are used for generation.
Testing & Traces	10	All 5 scenarios covered; traces clearly show decision flow; evaluation_results.md is thorough.

Viva Voce Guidelines

A viva will be conducted after submission. The viva carries implicit weight, if you cannot explain your code, marks for that section will be reduced regardless of submission quality.

Report submission

Students must submit two (2) PDF files along with the GitHub repository link:

#	PDF File	Contents
1	**Whole Project Report**	A single PDF covering the entire project built in labs.
2	**Part B Report**	A dedicated PDF for Part B (Self-RAG Agent) including architecture diagram, evaluation results, and all test case traces.